



Relatório de Qualidade — QA Completo

Solarize · Sistema de Gestão de Energia Solar

Gerado em 05/06/2026 às 21:30 UTC

1. Resumo Executivo

Métrica	Antes do QA	Após o QA	Status
Total de testes	71	82	MELHOROU
Testes passando	71 / 71	82 / 82	100%
Testes falhando	0	0	ZERO
Testes novos adicionados	—	11	ADICIONADO
Bugs críticos encontrados	—	0	ZERO
Gaps de negócio identificados	—	4	ATENÇÃO

Conclusão: O Solarize está **funcionalmente completo e estável** para a demo. O fluxo ponta-a-ponta (registro → contrato → geração → dashboard → PDF → auditoria) funciona corretamente e está coberto por testes de integração. Nenhum bug crítico foi encontrado. Os itens identificados são lacunas de negócio que precisam de decisão do time, não erros de implementação.

2. Cobertura por Endpoint

Endpoint	Método	Testes	Isolamento Org	Auth
POST /auth/register	POST	4	N/A	✓

POST /auth/login	POST	3	N/A	✓
GET /auth/profile	GET	3	Sim	✓
GET /contracts/	GET	4	Sim	✓
POST /contracts/	POST	7	Sim	✓
GET /contracts/{id}	GET	3	Sim (403)	✓
PUT /contracts/{id}	PUT	4	Sim (403)	✓
POST /generation/preview	POST	5	Sim (404)	✓
POST /generation/	POST	8	Sim (404)	✓
POST /generation/{contract_id}/audit	POST	8	Sim (404)	✓
GET /dashboard/landlord	GET	7	Sim	✓
GET /pdf/{generation_id}	GET	5	Sim (404)	✓
GET /health	GET	2	N/A	—

3. Novos Testes Adicionados (11 testes)

Arquivo	Teste	O que verifica
test_audit.py	test_audit_valid_chain	Cadeia de 3 gerações intacta — chain_valid = True
test_audit.py	test_audit_requires_auth	401 sem token
test_audit.py	test_audit_no_records_returns_404	404 quando contrato não tem gerações
test_audit.py	test_audit_nonexistent_contract_returns_404	404 para contract_id inexistente
test_audit.py	test_audit_org_isolation	Org B não audita contratos da Org A (404)
test_audit.py	test_audit_invalid_uuid_returns_422	422 para UUID malformatado
test_audit.py	test_audit_single_record	1 registro sem previous_hash — chain_valid = True

test_audit.py	test_audit_tampered_hash_detected	Hash adulterado no DB → chain_valid = False
test_generation_create.py	test_create_generation_duplicate_period_409	409 ao registrar mesma data duas vezes
test_services.py	test_cors_origins_strips_brackets	cors_origins_list remove brackets do .env
test_contratos.py	test_value_kwh_can_change_after_generation	Documenta gap: value_kwh mutável após geração

4. Riscos e Bugs Encontrados

[MÉDIO] RISK-001 value_kwh mutável após registros de geração existirem

Localização:	app/routers/contratos.py – PUT /contracts/{id}
Descrição:	O endpoint de atualização de contrato bloqueia corretamente a alteração de landlord_percentage após gerações existirem, mas não bloqueia a alteração de value_kwh . Como os PDFs são gerados dinamicamente usando a tarifa atual do contrato, alterar value_kwh retroativamente faz com que PDFs de gerações históricas calculem o valor financeiro com a tarifa nova — corrompendo o histórico.
Impacto:	PDFs históricos mostrarão valores financeiros incorretos após mudança de tarifa.
Recomendação:	Adicionar o mesmo bloqueio que já existe para landlord_percentage: se existirem registros de geração, rejeitar com 409 qualquer mudança em value_kwh.

[MÉDIO] RISK-002 Refresh Token sem endpoint de renovação

Localização:	app/routers/auth.py – POST /auth/login
Descrição:	O endpoint de login retorna um campo refresh_token na resposta, mas não existe nenhum endpoint POST /auth/refresh para usá-lo. Se o frontend implementar renovação de sessão usando este token, receberá um 404 e o usuário será deslogado inesperadamente após o token de acesso expirar (padrão: 24 horas).
Impacto:	Usuários deslogados após 24h sem possibilidade de renovação silenciosa de sessão.
Recomendação:	Implementar POST /auth/refresh que aceita o refresh_token e retorna um novo access_token, ou remover o campo refresh_token da resposta se a renovação não estiver planejada.

[BAIXO] RISK-003 Hash SHA-256 pode falhar na auditoria com precisão decimal > 4 casas

Localização:	database/models.py – calculate_hash() + EnergyGeneration.energy_kwh
Descrição:	O campo energy_kwh é declarado como Numeric(12, 4) no banco de dados (PostgreSQL armazena até 4 casas decimais), mas a função calculate_hash() formata o valor com :.6f (6 casas decimais). Se um usuário enviar um valor com mais de 4 casas decimais (ex: 1000.12345), o hash é calculado na criação com "1000.123450" mas o PostgreSQL armazena como "1000.1235". Na auditoria, o hash é recalculado com "1000.123500" — resultado diferente → auditoria reporta registro como adulterado incorretamente.

Impacto:	Falso positivo no endpoint de auditoria: registros legítimos reportados como adulterados em produção (PostgreSQL). Nos testes com SQLite este bug não manifesta.
Recomendação:	Ou: (a) truncar/arredondar energy_kwh para 4 casas no GenerationRequest antes de calcular o hash; ou (b) alinhar calculate_hash() para usar :.4f em vez de :.6f.

[BAIXO] RISK-004 end_date pode ser anterior a start_date em contratos	
Localização:	app/models/schemas.py – ContractCreate
Descrição:	O schema de criação de contrato aceita um campo end_date opcional, mas não valida que ele seja posterior ao start_date . É possível criar um contrato com end_date='2020-01-01' e start_date='2026-01-01' sem qualquer erro de validação.
Impacto:	Dados inconsistentes no banco. Pode causar confusão no frontend ao exibir vigência do contrato.
Recomendação:	Adicionar um @model_validator no ContractCreate que verifique if end_date and end_date < start_date: raise ValueError(...).

[INFO] INFO-001 Senha do seed não atende à política de registro	
Localização:	seed.py – _pwd.hash("solarize2026")
Descrição:	Os usuários criados pelo seed.py usam a senha " solarize2026 " (minúsculas, sem caractere especial). O endpoint POST /auth/register rejeita esta senha com 422 pois exige maiúscula + minúscula + número + caractere especial. O seed contorna isso inserindo o hash diretamente no banco, mas a senha exibida na demo não passaria no formulário de cadastro, o que pode confundir demonstradores.
Impacto:	Usuário de demo com senha fraca que não pode ser recriada pela interface.
Recomendação:	Atualizar seed.py para usar uma senha que atenda à política (ex: "Solarize2026@") e atualizar a documentação da demo.

5. Checklist de Funcionalidades

Funcionalidade	Status	Observação
Registro de usuário com validação de senha forte	PASS	Regex: maiúscula + minúscula + número + especial
CNPJ compartilhado cria uma única organização	PASS	Dois usuários com mesmo CNPJ entram na mesma org
Login com JWT — access + refresh token	PASS	Refresh token presente mas sem endpoint de uso
Proteção de rotas (401 sem token)	PASS	Todos os endpoints protegidos testados
Profile retorna dados do usuário correto	PASS	ID, nome, email, org_id, role
CRUD completo de contratos	PASS	Criar, listar, buscar, atualizar
Isolamento multi-org em contratos	PASS	403 para acesso cruzado
Validação de landlord_percentage ($0 < x \leq 1$)	PASS	422 para 0 e >1
Validação de value_kwh (> 0)	PASS	422 para 0 e negativos
Número de contrato único por organização	PASS	Orgs diferentes podem usar mesmo número
landlord_percentage bloqueado após geração	PASS	409 se existir geração no contrato
value_kwh mutável após geração existir	GAP	Risco RISK-001: corromperia PDFs históricos
Preview de geração (cálculo sem salvar)	PASS	saved=false, fórmula correta $E \times T \times P \times 0.95$
Criação de geração com hash chain	PASS	previous_hash encadeado corretamente
Duplicata de período retorna 409	PASS	Mesma data + mesmo contrato rejeitado
Cálculo financeiro ROUND_HALF_UP	PASS	38500 kWh → R\$ 9.326,63 ✓
Auditoria de cadeia de hashes	PASS	Detecta adulteração, valida cadeia completa
Isolamento de auditoria por organização	PASS	Org B não acessa cadeia da Org A

Dashboard — mês atual com hash	PASS	Hash da última geração do mês incluído
Dashboard — série histórica (max 3 meses)	PASS	Exatamente 3 meses, mais antigo ao mais novo
Dashboard agrega múltiplos contratos	PASS	Soma KWh e valor de todos os contratos da org
PDF com dados reais do banco	PASS	Locador, locatário, tarifa, hash SHA-256
Isolamento de PDF por organização	PASS	404 para geração de outra org
Cabeçalho Content-Disposition no PDF	PASS	filename=solarize_YYYY-MM.pdf
CORS com brackets no .env	PASS	cors_origins_list strip correto
Rate limiting desabilitado nos testes	PASS	RATE_LIMIT_ENABLED=false no conftest
Health check	PASS	Status: OK, Version: 1.0
Hash integrity com >4 casas decimais	RISCO	Risco RISK-003: mismatch em produção (PostgreSQL)
Refresh token — endpoint de renovação	GAP	Risco RISK-002: campo presente sem endpoint
end_date >= start_date no contrato	GAP	Risco RISK-004: sem validação no schema